

Expressivity of Parametrised Quantum Circuits: Dependence on the Spectrum of the Generator

Florian Velten
s2124432

21 May 2026

Contents

1 Introduction	1
2 Models and Target Functions	3
2.1 Circuit Architecture	3
2.2 Target Functions	4
3 Results and Discussion	5
3.1 Fourier Coefficient Analysis	5
3.2 Fitting the Models	6
3.3 Discussion	6
4 Conclusion and Outlook	8
A Large Figures and Tables	9

1 Introduction

Parametrised quantum circuits are the object of interest in the field of quantum machine learning. They can be used to approximate a target function $g : \mathbf{R} \rightarrow \mathbf{R}$ as follows. First, an initial quantum state of N qubits is prepared, most often $|0\rangle$. Next, a sequence of unitaries is applied, which either encode the input data or are determined by trainable parameters. Finally, the expectation value of some observable is measured, which is the output of the system. The process of using this output, comparing it to the target function, and then optimising the parameters to get closer to g is the prototypical optimisation loop of variational quantum algorithms.

It is, of course, interesting to investigate what classes of functions may be reached when using a certain type and sequence of encoding and trainable layers, a property that may be referred to as *expressivity*. Some research into this has been done, for example by Schuld, Sweke and Meyer [2]. In this paper, the authors investigate the expressivity of an often-used architecture, which I will explain now. For an illustration, see Figure 1, which is taken directly from Figure 1 of [2].

The encoding layers are given by the operation $S(x) = \exp(ixH)$, where H is a fixed Hermitian operator called the *generator*, and the trainable layers are represented as arbitrary unitary operations denoted $W^{(k)}$, which in practice are chosen to be so called *ansatze*. These two layers are applied in an alternating fashion, starting and ending with a trainable layer and containing L encoding layers.

Schuld, Sweke and Meyer show that this particular architecture is *approximately universal*, in the sense that any target function can be approximated to arbitrary precision, given that the number of qubits N , the number of encoding layers L , and the generator for the encoding layers can be chosen freely. There is an

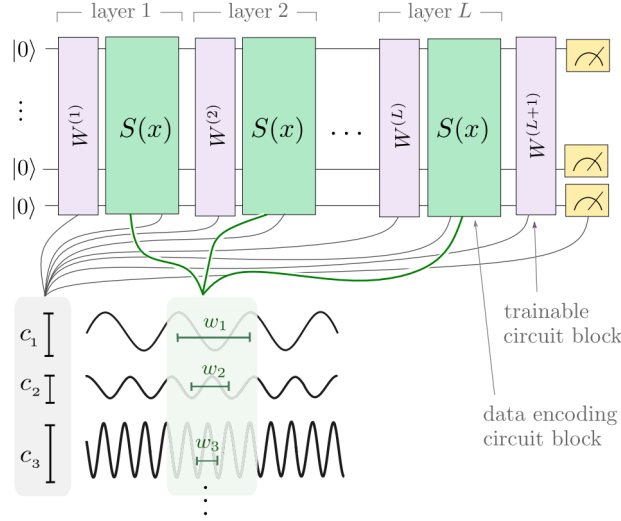


Figure 1: Copy of Figure 1 in [2]. To the initial state, a sequence of alternating encoding and trainable layers is applied. The encoding layers determine the Fourier series frequency spectrum of the output, while the coefficients depend on all layers and the measurement.

important caveat, however: for this statement, the authors assume that each trainable layer may become any arbitrary unitary operation. This is, of course, not feasible in practice, but the statement nonetheless validates the use of this architecture.

Going into more detail, the main result that enables approximate universality is the observation that this architecture yields a *partial Fourier series* as approximating function, no matter the choice of generator, trainable ansatz or observable. Symbolically, if

$$f_{\theta}(x) = \langle 0 | U(x, \theta)^{\dagger} M U(x, \theta) | 0 \rangle$$

is the approximating function, where

$$U(x, \theta) = W^{(1)}(\theta) S(x) W^{(2)}(\theta) S(x) \dots W^{(L)}(\theta) S(x) W^{(L+1)}(\theta)$$

is the alternating sequence of trainable and encoding layers, then we have

$$f_{\theta}(x) = \sum_{\omega \in \Omega} c_{\omega}(\theta) e^{i\omega x},$$

the partial Fourier series. Here, the set of frequencies Ω is completely determined by the generator H , hence, importantly, does not depend at all on the trainable layers or the observable. In particular, if $\lambda_1, \dots, \lambda_{2^N}$ are the (not necessarily distinct) eigenvalues of H , we have

$$\Omega = \left\{ \sum_{a=1}^L \lambda_{j_a} - \lambda_{k_a} : 1 \leq j_a, k_a \leq 2^N \right\}.$$

The above seems to imply that the here described circuit architecture should easily be able to learn any 2π -periodic function whose frequencies are reachable by choosing an appropriate generator. However, it is important to note that the choice of trainable ansatz is essential in determining what *coefficients* are attainable. Moreover, it is unclear how well the architecture does in approximating non-periodic functions when the number of qubits is small. In conclusion, there are two questions that I have investigated in this project:

1. How much more expressive are circuits with “better” generators, i.e. with generators for which the set Ω is larger?

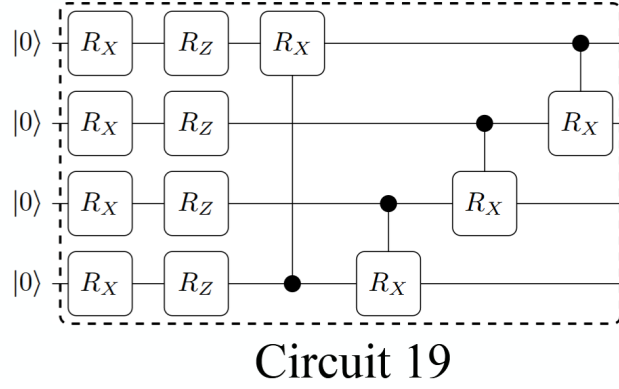


Figure 2: Circuit 19 in [3], taken directly from Figure 2 in said reference. Each rotation, controlled or not, corresponds to one trainable parameter.

2. How well can such circuits approximate different classes of functions, even non-periodic ones?

The expected answer to these questions seem somewhat clear. A richer set of attainable frequencies will of course make the circuit much more expressive, all else being equal. In fact, once the target function contains frequencies that the circuit cannot attain, I expect the circuit to perform quite badly. Furthermore, the function classes for which the architecture does well will be those whose members have an important oscillatory component. If no such component is present, good approximations will likely require far larger numbers of qubits and more encoding layers.

2 Models and Target Functions

2.1 Circuit Architecture

There are a lot of hyperparameters that may be tweaked and whose effect on the result may be investigated. However, given that I had about one month's worth of non-full-time work to put into this project, I decided to keep a lot of hyperparameters fixed, so that I could address the two questions above in a systematic way.

First, I chose $N = 3$ qubits and $L = 1$ encoding layers. This may seem like a very limiting choice, but it turns out that it is not. In fact, as I will explain below, the set of attainable frequencies Ω can be surprisingly large for well-chosen generators, which do not have to be too complex, either.

Next is the question of what trainable ansatz to use. Based on the observations of Schuld, Sweke and Meyer [2] in Section III.C on the Fourier coefficient distribution, I looked for a relatively simple ansatz with a nonetheless rich set of possible coefficients. I found it as circuit 19 in [3], which Strobl et al. [4] showed to be close to optimal in the sense that the coefficients of different frequencies are very close to uncorrelated. This circuit, which I will henceforth refer to as *the* ansatz or trainable layer, can be found in Figure 2. It consists of two layers of parametrised single-qubit rotations, which is followed by a sequence of single-controlled single-qubit rotations arranged in a cascade. It is easily generalised to any number of qubits N and in the general case has $3N$ trainable parameters. Thus, the total circuit has $3N(L + 1) = 18$ parameters, each of which can be chosen from the interval $[0, 2\pi]$.

Closely related to the choice of ansatz is the choice of observable. There are, of course, many options, but I chose a simple one which I expected would not limit the model: the total magnetisation, which is given by the sum of Pauli Z operators applied to each qubit, namely $M = \sum_{i=1}^N Z_i$.

But the most interesting choice for this project is, of course, what generators to use. I wanted to have a good spread in complexity, from very simple to very rich, to be able to make good comparisons. After some thought, I settled on the four given in Table 1. Note that that all attainable frequencies are integers; this is intentional. In fact, I do not expect choices of generators with algebraically independent eigenvalues to

Name	Generator	Attainable frequencies
Single-qubit Pauli	$\frac{1}{2}Z_1$	$\Omega = \{0, \pm 1\}$
Triple-qubit Pauli	$\frac{1}{2}(Z_1 + Z_2 + Z_3)$	$\Omega = \{0, \pm 1, \pm 2, \pm 3\}$
Optimal sum of Paulis	$\frac{1}{2}(Z_1 + 3Z_2 + 9Z_3)$	$\Omega = \{0, \pm 1, \dots, \pm 13\}$
Engineered spectrum	$\text{diag}(0, 1, 4, 9, 15, 22, 32, 34)$	Integers between -34 and 34 (inclusive), but not $\pm 16, \pm 20, \pm 24, \pm 26, \pm 27$, or ± 29 .

Table 1: The four chosen generators and their corresponding attainable frequencies. Note that Ω always contains 0 and is closed under negation. Also note that Ω is contiguous for the first three generators, but not for the ‘Engineered spectrum’ generator.

be feasible to implement on quantum computers, at least not in the near future, and I do not expect this to be necessary or desirable in any case. Moreover, if all eigenvalues are algebraically dependent, one can always rescale to ensure that all of them are integer multiples of some basic frequency ω_0 , which then only requires a rescaling of the domain of the approximating function to recover an analogous case to what I am describing here. This was already pointed out by Schuld, Sweke and Meyer [2] in Appendix B.

Also note that all generators are diagonal; this is also intentional. Any non-diagonal generator may be diagonalised using a simple basis change. In the ideal case, the basis change unitary can simply be absorbed into the trainable layer which are assumed to be able to represent any unitary anyway. In the non-ideal case of a particular ansatz, it may make a difference, but I do not expect it to make a large one. In any case, investigating this would increase the scope of the project so much that I would not have had time for it.

The Single-qubit Pauli generator was chosen for its simplicity, as a sort of ‘control group’. Since the set of attainable frequencies is so small, I expect the expressivity to be severely limited; in fact, only simple sinusoidal oscillations with period 2π will be reachable target functions. The Triple-qubit Pauli generator is a more complex extension of this basic case, which should, in direct comparison, perform a lot better.

The Optimal sum of Paulis generator was engineered to induce the largest possible but still contiguous set Ω . The coefficients were determined using a simple script that searches for the largest induced set Ω by brute force iterating over all integer linear combinations. Since Ω is large, but still contiguous, I expect this choice of generator, which should still be easy to implement on real hardware, to perform better than the previous two.

And finally, the Engineered spectrum generator was chosen to maximise the cardinality of Ω . Its diagonal entries were chosen according to the optimal Golomb ruler¹ of order $8 = 2^3 = 2^N$. This generator may perform best in the sense that it should be able to approximate the most target functions the closest, but the non-contiguity of Ω , as well as the small number of trainable parameters with respect to the number of Fourier coefficients may pose problems.

2.2 Target Functions

A natural way to compare these four generators with each other is to take a random instantiation of a model with one generator, i.e. a random selection of parameters, as target function and try to fit the other models to it. With each generator, I generated three random realisations. I expect that the models will be able to successfully fit a random target function generated using any of the generators presented *above* it in Table 1, but not using one below it. This seems like a good way to directly compare them against each other.

On the other hand, it is perhaps even more interesting to look at other classes of functions to determine how general the models are. For this, I first generated random truncated Fourier series by directly generating the coefficients from the complex unit disc. The restriction to the unit disc stems from the observation that the reachable coefficients are apparently contained within the unit disc, as can be seen in Figure 3; more on this below in Section 3.1. Three such target functions were generated, with the frequency cut-offs somewhat

¹see ‘Golomb ruler’ on Wikipedia

arbitrarily set at 1, 3, and 10, respectively. My goal here was to have one control target, which all models should be able to fit to, one which separates the simplest generators from the more complex ones, and one which should be reachable in theory by the complex generators, but which will test their limits.

Next, I chose a class of functions that have a strong oscillatory component, but which is not a pure Fourier series. This is exhibited by decaying oscillations; the general expression for a potential target, with parameters κ and λ , is $g(x) = \exp(-\kappa\lambda x) \cos(\lambda x)$, a type of function I encountered some time ago in [1]. After some experimentation, I found that the coefficient pairs (κ, λ) equal to (0.05, 3), (0.05, 8) and (0.02, 20) should satisfy the same criteria as described above for the random truncated Fourier series' cut-off frequencies.

Finally, I chose a set of functions with no (obvious) oscillatory component at all. On the one hand, I chose three randomly generated polynomials, one per degree 1, 2, and 3, and on the other hand I chose the 'special' functions $\sqrt{x}$, $\exp(x)/100$, and $\sigma(x) = 1/(1 + e^{-x})$, which seemed to me to be commonplace and thus interesting. The factor of $\frac{1}{100}$ for the exponential function was chosen to make it easier for the models to fit to by limiting its range on the interval $[0, 2\pi]$; in practice, such a simple rescaling can always be done as preprocessing and then undone after fitting.

In conclusion, I fitted all four models to each of the 24 target functions: three random instantiations of each of the four models themselves, three random truncated Fourier series, three decaying oscillations, three random polynomials, and three 'special' functions. The resulting graphs and quality-of-fit quantifications can be found in Appendix A and will be explained the next section.

3 Results and Discussion

The code I used to do the analysis and produce the plots and tables can be found on my GitHub page². I used the PennyLane framework for circuit creation and optimisation.

3.1 Fourier Coefficient Analysis

The first bit of analysis I did was to map out the Fourier series coefficients that each generator produces together with the ansatz and the observable. For each model, I generated 100 sets of random parameters and then Fourier transformed the resulting functions $[0, 2\pi] \rightarrow \mathbf{R}$ to obtain the coefficients. For each frequency I then scattered the 100 coefficients per frequency on the complex plane, as can be seen in Figure 3. Moreover, I calculated the standard deviation over the 100 coefficients of each frequency, the results of which can be seen in Figure 4. This was mainly to confirm that the circuit generation worked as expected.

And it did: we can clearly see in the figures that precisely those coefficients are non-zero that correspond to the frequencies that are contained within the respective sets Ω . For the other two generators, which induce more than nine frequencies, Figure 3 shows only a selection of the coefficients, but it is clearly visible in Figure 4 that precisely the coefficients up to frequency 13 are non-zero for the Optimal sum of Paulis generator and that exactly the expected frequencies of the Engineered spectrum generator are missing. Note that for this last generator, I already included the graph for another architecture with more trainable layers; see Section 3.3.3 for details.

Apart from confirming that my code works, these figures also allow for interesting observations. For one, the coefficient of frequency 0 is always real; this is as expected. Also, the overwhelming majority of coefficients is contained within the unit disc, and the spread is much more concentrated around 0 for most non-zero frequencies than would be desirable. This will almost certainly have a large impact on the ability of the models to learn different function classes, since not all Fourier coefficient values are, apparently, equally reachable.

²https://github.com/RealMrAnonymous/aqa_generating_set_expressivity

3.2 Fitting the Models

The optimisation process was set up as follows. First, for each model, a fixed set of collocation points was generated. Since we are in principle generating a truncated Fourier series, it is important to consider the Nyquist-Shannon sampling theorem, which states that a signal which contains no frequencies above ω_0 can be perfectly reproduced by its values at collocation points which are spaced no more than π/ω_0 apart. Thus, the domain $[0, 2\pi]$ should be split into at least $2\omega_0$ portions of roughly the same size. I chose to pick a little more than $2.5\omega_0$ collocation points for all models, namely:

- 5 points for the Single-qubit Pauli generator,
- 10 points for the Triple-qubit Pauli generator,
- 30 points for the Optimal sum of Paulis generator, and
- 75 points for the Engineered spectrum generator.

As loss function, I chose the mean squared error over the collocation points. This choice is somewhat arbitrary, but it seemed the simplest to implement and has nice properties like being differentiable (unlike mean absolute error). As optimiser, I used the Adam implementation of PennyLane with a learning rate of 0.01. This choice of learning rate is also somewhat arbitrary, but reasonable. Finally, I ran 1000 training steps for each pair of model and target function.

The visualised results of the training can be seen in Figures 5 and 6. In each sub-figure, a row shows the results of fitting all four models to one target function as the loss values at each training step (left column) and the final result against the target function (right column). Moreover, Tables 2 and 3 display the numerical results for each training run. For each combination of model and target function, the values displayed are the final mean squared error measured at the collocation points, i.e. the final loss value, as well as the mean squared error measured at 1000 equally spaced points on the domain $[0, 2\pi]$, which is a decent measure of how close the fit is to the target function.

It is important to note that not all numerical values should be compared. Per target function, comparisons are fine, of course, but different target functions, or perhaps even different instances of a randomly generated target function, can have significantly different ranges, so that the numerical values may be slightly skewed.

3.3 Discussion

3.3.1 Models versus Models

Let us first consider the target functions given by randomly generated model instances (Figure 5 and Table 2). In general, the results somewhat confirm the expectations, although there is one big exception: it seems that the Engineered spectrum model failed to learn the target function at all in some cases. More on that later; for now, it will be disregarded wherever it failed.

All models managed to comfortably learn both the Single-qubit and the Triple-qubit Pauli generated instances. In fact, it is surprising how well the Single-qubit Pauli model did in learning the Triple-qubit Pauli generated functions, but this may have been luck in that the high frequency components almost vanish.

When it comes to the targets generated using the other two models, the Single- and triple qubit Pauli models fail to learn anything more than the lowest frequency components. In fact, it is clear that aliasing has occurred in most cases, since the test error is far larger than the training error, as can be seen in Table 2. Only the Optimal sum of Paulis model manages to perfectly fit its own target functions and follows the Optimal spectrum generated target functions the closest, although not very well.

There are some comments to be made about the loss value graphs, as well. For one, the loss values plateau after a relatively small amount of training steps in several cases. Sometimes, they then suddenly start decreasing again, after which another plateau is hit. It may be the case that this is a good example of the ‘barren

plateau' property that many gradient-based quantum machine learning models exhibit. Such barren plateaus may be dealt with, at least in the case of small models as we have here, for example by increasing the total number of training steps (another order of magnitude, for a total of 10,000, is still reasonable, two may be possible with faster computers), which in practice just means traversing the entire plateau. Another option is to increase the learning rate (e.g. by an order of magnitude to 0.1), and perhaps make it dynamic, i.e. slowly decrease it with the number of steps taken. This would mean that early plateaus are traversed more quickly. Unfortunately, I had time to test neither option, so let us move on to the other target functions.

3.3.2 Models versus Other Target Functions

See Figure 6 and Table 3. The results here also somewhat confirm the expectations. The models do well to learn the low frequency random truncated Fourier series (Figure 6a), the low and medium frequency decaying oscillation (Figure 6b), the random degree 1 polynomial (Figure 6c), and the sigmoid function (Figure 6d). In fact, these target functions are different from the rest in that their low frequency components are large compared to the high frequency ones, which is less the case for the other target functions. Moreover, the models really struggle with the non-periodic target functions, which is very much expected, perhaps even inevitable. This may be explained simply by noting that the Fourier series of non-periodic functions have relatively far larger high-frequency coefficients, which the models simply cannot accommodate since they inherently generate periodic functions with a relatively low maximum frequency.

Next, it is noteworthy that even though the Optimal sum of Paulis and the Engineered spectrum models *should* have a large enough frequency range to learn all truncated Fourier series target functions, they fail already at the one with maximum frequency 3. This may be due to the fact that the range of reachable *coefficients* is too limited to represent all possible combinations.

I would also like to note that the Optimal sum of Paulis model is almost always the best performing one, as measured by the Test MSE, and when it is not, it is very close. This could mean that this model hits a certain sweet spot between the number of reachable frequencies and the number of reachable coefficients, for the given number of trainable parameters. Indeed, I expect that more trainable parameters means more reachable coefficients, hence better expressivity; this is a possible direction of further inquiry, for which I only had limited time.

3.3.3 More Trainable Layers

Finally, there is something that could explain the prevalent failure of the Engineered spectrum model to learn entirely. In particular, changing one parameter may have an effect on so many coefficients for this model, that the training landscapes become too difficult to navigate with the hyperparameters I used here. Thus, I expected that adding more parameters, i.e. more trainable layers, would improve performance of this model. However, this expectation turned out to be wrong.

Indeed, when I added two more variational layers (one before, one after the encoding layer) to the Engineered spectrum model and did some tests³, it was confirmed that the model does not at all perform better. To investigate why, I compared the spread of the Fourier coefficients of this new architecture with more parameters with the original one; see Figure 4 (bottom right). We can see here that the standard deviation, and hence the spread, of the coefficients is somewhat improved, but by no means as much as I had hoped it would be. In particular, no single coefficient spread comes close to the spread of the coefficients in the Optimal sum of Pauli model that correspond to frequencies 1, 3, and 9.⁴

In conclusion, while it is not clear *why* the coefficient spread does not improve when adding more trainable layers to Engineered spectrum model, this shows that we cannot expect better performance when compared to the original model.

⁴The results of these tests are omitted in this report, but can still be found in the code on GitHub; see an earlier footnote for the link.

4 Conclusion and Outlook

As stated in the Introduction, I have investigated two questions in this project: how much more expressive are circuits with rich Ω , and how do these circuits perform on different classes of functions. It seems that I was able to answer both questions.

It is clear from Figure 5 that a richer set of attainable frequencies indeed improves expressive power, but when the set gets very large, training becomes far more difficult. As we have seen, only maximising for size of Ω disregards the importance of the coefficients: if the range of realisable *coefficients* small, and it seems that it becomes smaller the larger the size of Ω , expressive power is again limited.

Unfortunately, the models have booked limited success when it comes to target functions of different types. Especially functions with very large variations and/or no significant periodic component pose a big problem.

Thus, we can conclude that these models are good for learning periodic functions in which the lowest frequency components dominate, and that there is a trade-off to be considered between the number of attainable frequencies and the spread of realisable coefficients.

However, my investigations here are far from comprehensive, and a lot of questions remain, two of which are: “Is it possible to quantify the trade-off between frequency range and coefficient spread?” and “Does coefficient spread improve in circuits with more layers and/or with more qubits?”

Furthermore it would be interesting to see if there are ways to improve the performance of these models without changing the architecture. This could be done by mitigating the problem of enforced periodicity by ignoring the edges of the training domain. To be more precise, what happens if training and testing is only done using points in, for example, the interval $[\varepsilon, 2\pi - \varepsilon]$ for some non-zero value of ε ?

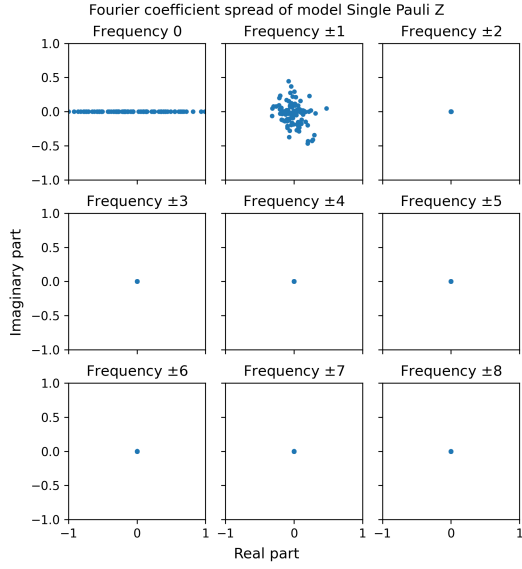
Or perhaps this problem can be avoided entirely by allowing *preprocessing* of the input data. This has been done by, for example, Kyriienko, Paine and Elfving [1], who use a data encoding layer of the form $\exp(i \arccos(x)H)$. The addition of the arccosine means that the model is not learning the target function $g(x)$ directly, but rather $g(\cos\theta)$, which is clearly periodic.

In short, there are still a lot of open questions that are very much worth investigating to further our knowledge of quantum machine learning.

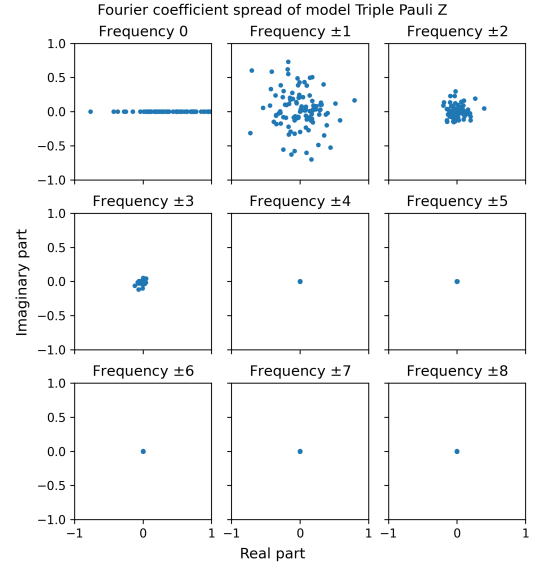
References

- [1] Oleksandr Kyriienko, Annie E. Paine and Vincent E. Elfving. ‘Solving nonlinear differential equations with differentiable quantum circuits’. en. In: *Physical Review A* 103.5 (May 2021). ISSN: 2469-9926, 2469-9934. DOI: 10.1103/PhysRevA.103.052416. URL: <https://link.aps.org/doi/10.1103/PhysRevA.103.052416> (visited on 16/12/2025).
- [2] Maria Schuld, Ryan Sweke and Johannes Jakob Meyer. ‘Effect of data encoding on the expressive power of variational quantum-machine-learning models’. en. In: *Physical Review A* 103.3 (Mar. 2021). ISSN: 2469-9926, 2469-9934. DOI: 10.1103/PhysRevA.103.032430. URL: <https://link.aps.org/doi/10.1103/PhysRevA.103.032430> (visited on 21/04/2026).
- [3] Sukin Sim, Peter D. Johnson and Alan Aspuru-Guzik. ‘Expressibility and entangling capability of parameterized quantum circuits for hybrid quantum-classical algorithms’. In: *Advanced Quantum Technologies* 2.12 (Dec. 2019). arXiv:1905.10876 [quant-ph]. ISSN: 2511-9044, 2511-9044. DOI: 10.1002/qute.201900070. URL: <http://arxiv.org/abs/1905.10876> (visited on 30/04/2026).
- [4] Melvin Strobl et al. *Fourier Fingerprints of Ansatzes in Quantum Machine Learning*. arXiv:2508.20868 [quant-ph]. Aug. 2025. DOI: 10.48550/arXiv.2508.20868. URL: <http://arxiv.org/abs/2508.20868> (visited on 30/04/2026).

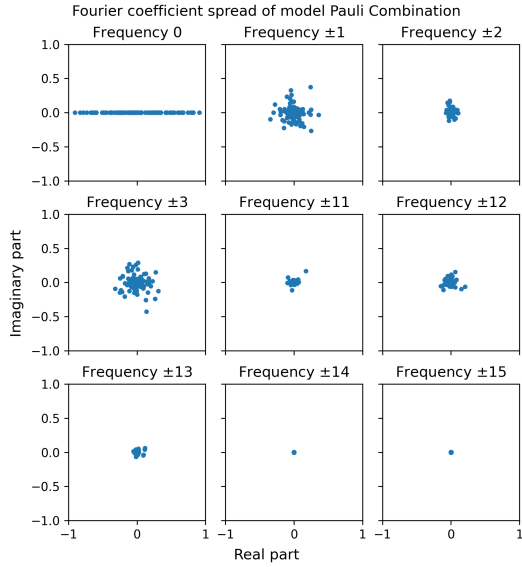
A Large Figures and Tables



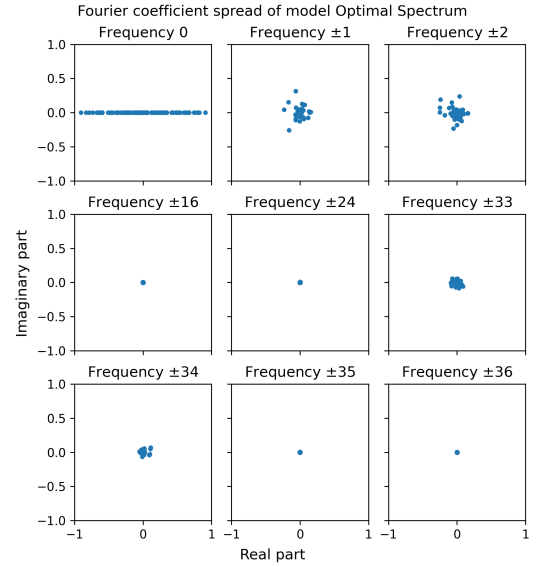
(a) Generator: Single-qubit Pauli



(b) Generator: Triple-qubit Pauli



(c) Generator: Optimal sum of Paulis. Note that not all frequencies are displayed.



(d) Generator: Engineered spectrum. Note that not all frequencies are displayed.

Figure 3: Fourier coefficients of 100 random instances of the four models, plotted on the complex plane.

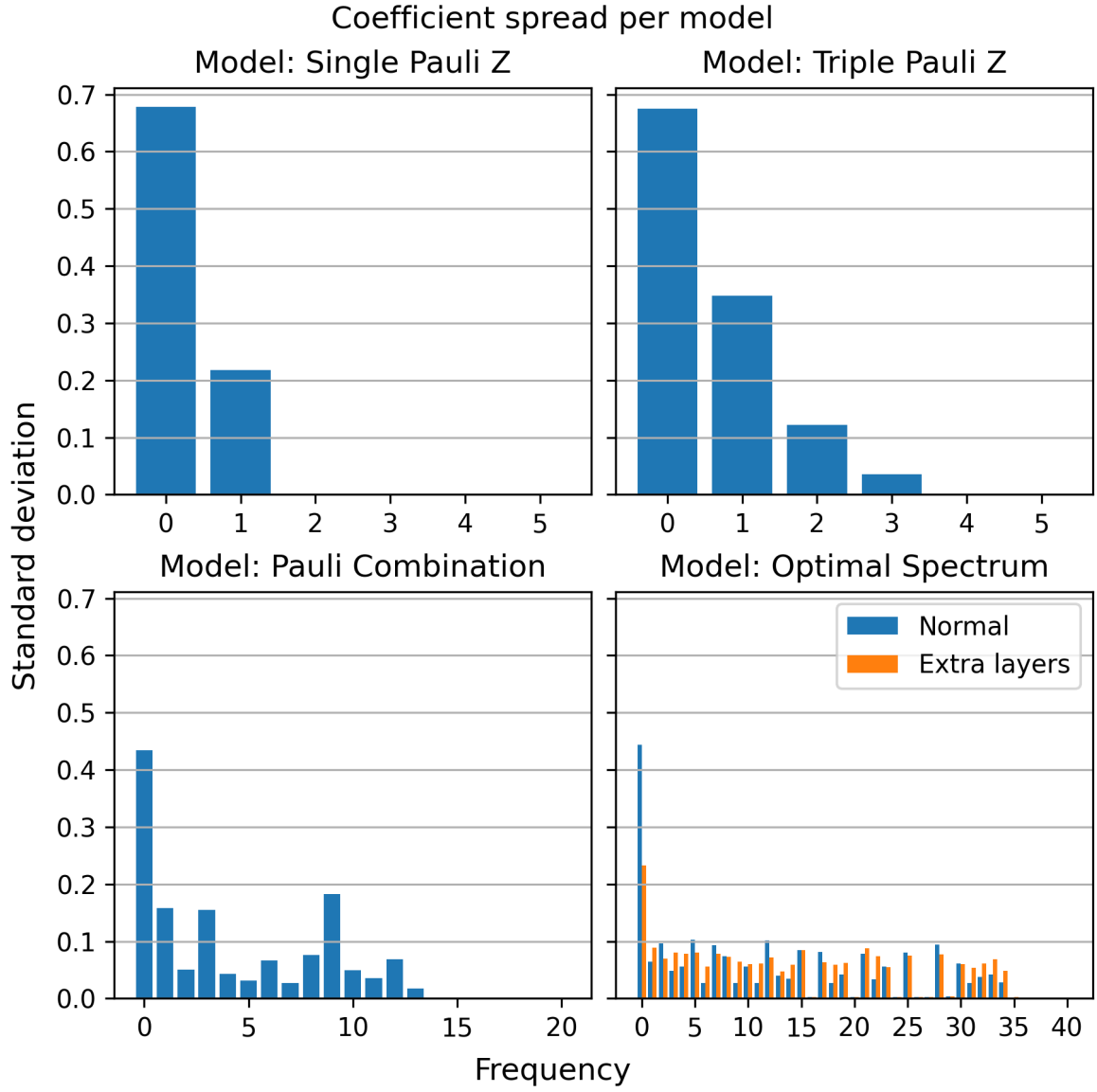
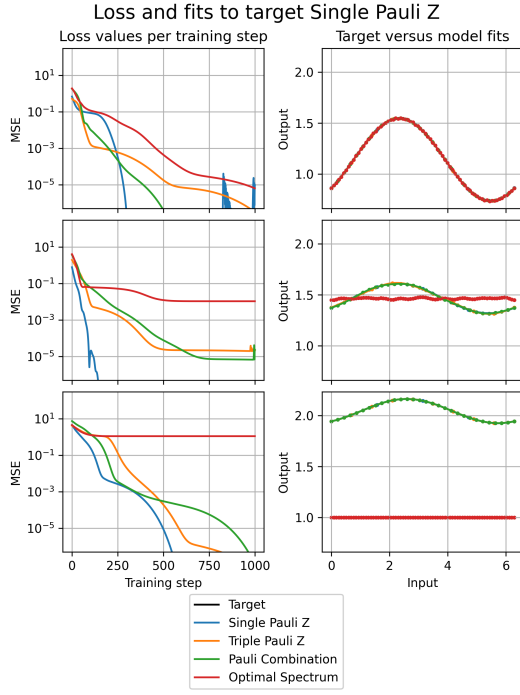
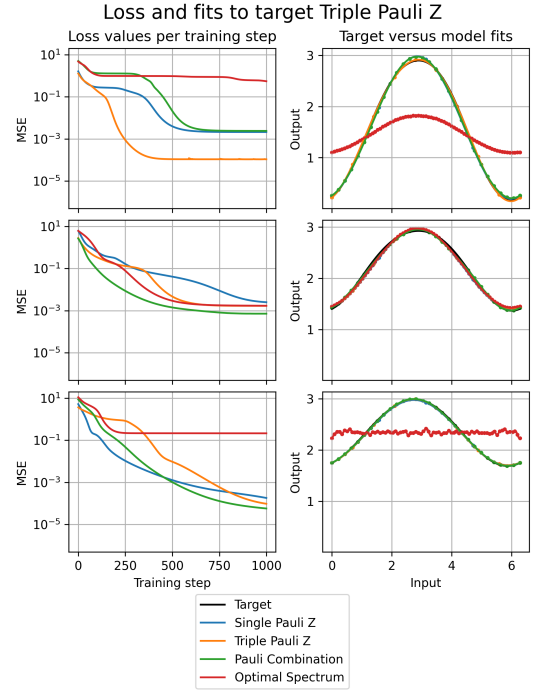


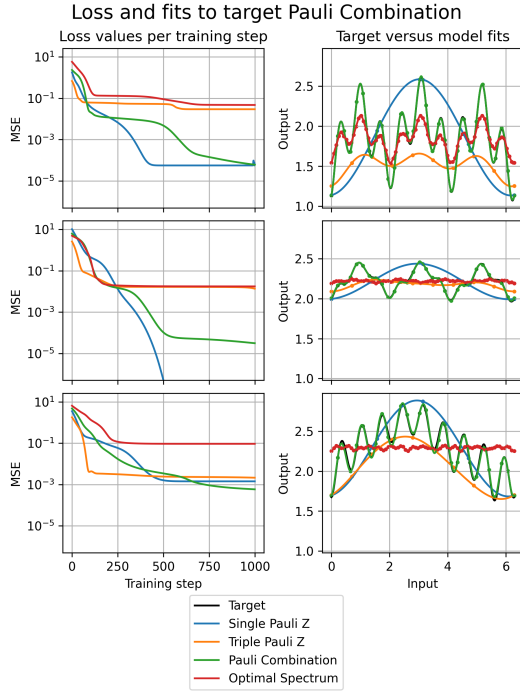
Figure 4: Standard deviations per Fourier coefficient of 100 random instances of the four models. For the model Engineered spectrum (lower right), a version with an extra layer is included; see Section 3.3.3.



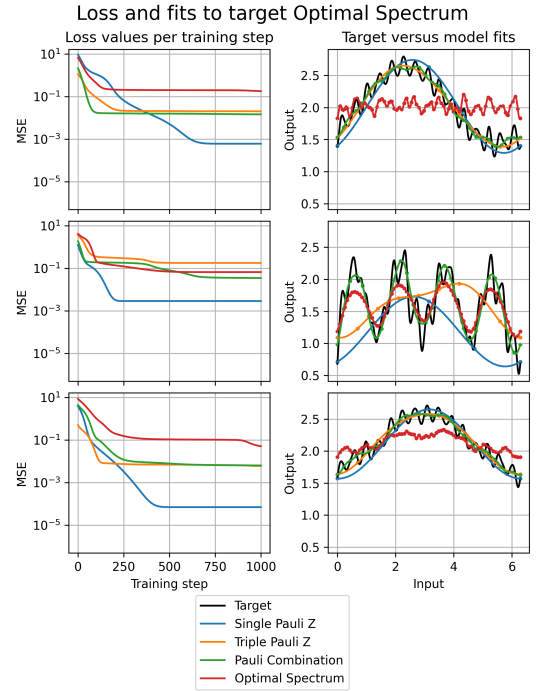
(a) The four models fitted to three random instances of the Single-qubit Pauli generator. In the second and third rows, only the red graph (Engineered spectrum) deviates significantly from the target.



(b) The four models fitted to three random instances of the Triple-qubit Pauli generator. In the first and third rows, only the red graph (Engineered spectrum) deviates significantly from the target.



(c) The four models fitted to three random instances of the Optimal sum of Paulis generator. The target function is hidden behind the green Optimal sum of Paulis graph.

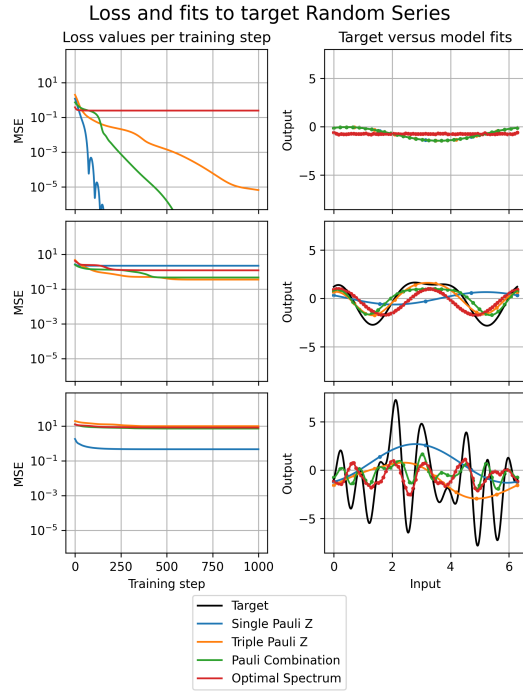


(d) The four models fitted to three random instances of the Engineered spectrum generator.

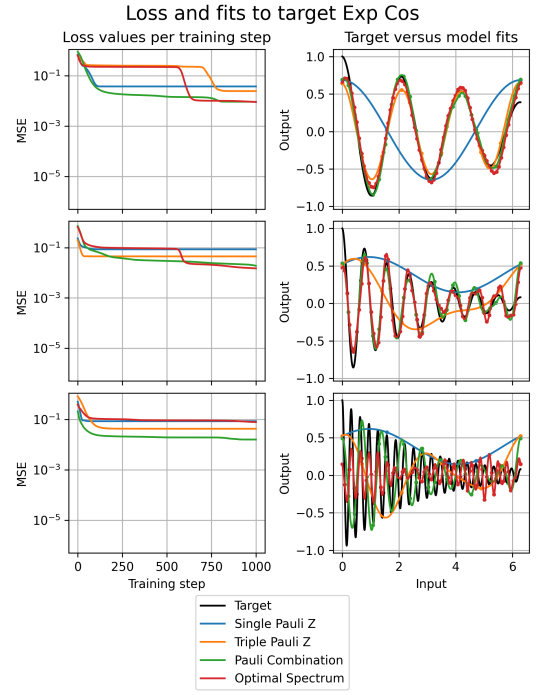
Figure 5: Results of fitting the four models against random instances of those same models. In each of the four sub-figures, one row contains the loss function at each training step in the left column, and the actual fitted function compared to the target function in black in the right column. Note that the target function is sometimes not visible because it is hidden behind a very well fitted model.

Target: Single-qubit Pauli	Instance 1		Instance 2		Instance 3	
	Train	Test	Train	Test	Train	Test
Single-qubit Pauli	0	0	0	0	0	0
Triple-qubit Pauli	0	0	0	0	0	0
Optimal sum of Paulis	0	0	0	0	0	0
Engineered spectrum	0	0	0.011	0.011	1.092	1.094
Target: Triple-qubit Pauli	Instance 1		Instance 2		Instance 3	
	Train	Test	Train	Test	Train	Test
Single-qubit Pauli	0.002	0.003	0.003	0.002	0	0
Triple-qubit Pauli	0	0	0.002	0.002	0	0
Optimal sum of Paulis	0.002	0.002	0.001	0.001	0	0
Engineered spectrum	0.544	0.539	0.002	0.002	0.213	0.213
Target: Optimal sum of Paulis	Instance 1		Instance 2		Instance 3	
	Train	Test	Train	Test	Train	Test
Single-qubit Pauli	0	0.336	0	0.038	0.001	0.078
Triple-qubit Pauli	0.029	0.216	0.013	0.017	0.002	0.094
Optimal sum of Paulis	0	0	0	0	0.001	0.001
Engineered spectrum	0.047	0.046	0.017	0.017	0.093	0.090
Target: Engineered spectrum	Instance 1		Instance 2		Instance 3	
	Train	Test	Train	Test	Train	Test
Single-qubit Pauli	0.001	0.034	0.003	0.477	0	0.020
Triple-qubit Pauli	0.020	0.015	0.177	0.244	0.006	0.011
Optimal sum of Paulis	0.015	0.014	0.035	0.036	0.006	0.009
Engineered spectrum	0.178	0.177	0.066	0.064	0.051	0.050

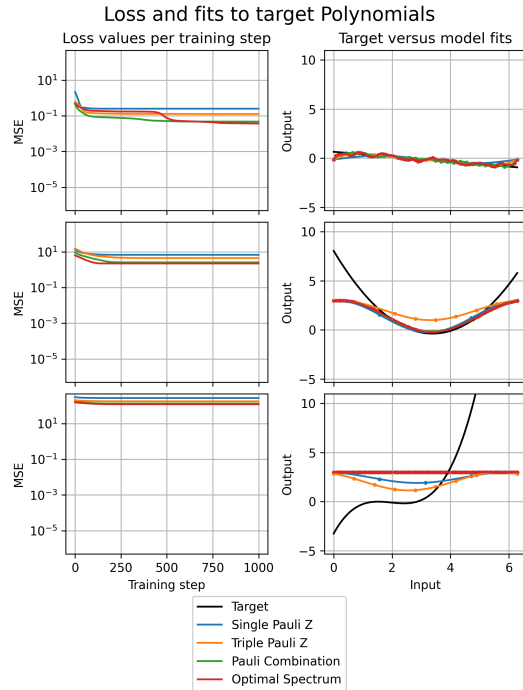
Table 2: Numerical results of fitting each model to three random instances of said models. For each instance, the column ‘Train’ displays the final mean squared error of the model with the target instance measured at the collocation points. The column ‘Test’ displays the final mean squared error measured at 1000 equally spaced points on the domain $[0, 2\pi]$. All values are rounded to three decimal places, so that value 0 means < 0.0005 .



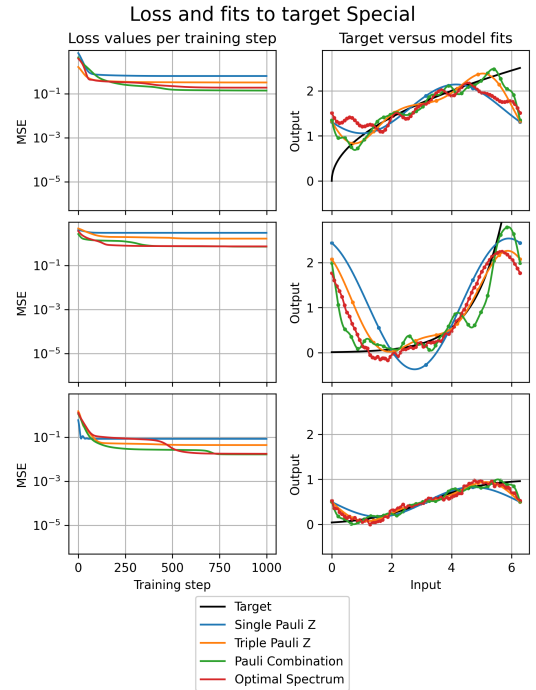
(a) The four models fitted to three randomly generated truncated Fourier series, with maximal frequencies 1 (top row), 3 (middle row) and 10 (bottom row). The target function in the top row is hidden behind the green Optimal sum of Paulis graph, which also hides the blue and yellow graphs.



(b) The four models fitted to three instances of the decaying oscillation function $\exp(-\kappa\lambda x) \cos(\lambda x)$, with parameters (κ, λ) equal to $(0.05, 3)$ in the top row, $(0.05, 8)$ in the middle row, and $(0.02, 20)$ in the bottom row. The target function is hidden behind the green Optimal sum of Paulis graph.



(c) The four models fitted to three randomly generated polynomials, of degrees 1 (top row), 2 (middle row) and 3 (bottom row).



(d) The four models fitted to the functions $\sqrt{x}$ (top row), $\exp(x)/100$ (middle row), and $\sigma(x)$ (bottom row), where $\sigma(x) = (1 + \exp(-x))^{-1}$.

Figure 6: Results of fitting the four models against other target functions. In each of the four sub-figures, one row contains the loss function at each training step in the left column, and the actual fitted function compared to the target function in black in the right column.

Target: Random series	Max freq. 1		Max freq. 3		Max freq. 10	
	Train	Test	Train	Test	Train	Test
Single-qubit Pauli	0	0	2.241	3.045	0.470	13.25
Triple-qubit Pauli	0	0	0.036	0.362	10.05	10.07
Optimal sum of Paulis	0	0	0.471	0.480	7.312	7.557
Engineered spectrum	0.248	0.248	1.219	1.234	8.323	8.427
Target: Decaying oscillation	(0.005, 3)		(0.005, 8)		(0.002, 20)	
	Train	Test	Train	Test	Train	Test
Single-qubit Pauli	0.038	0.444	0.086	0.274	0.086	0.272
Triple-qubit Pauli	0.025	0.017	0.045	0.194	0.043	0.186
Optimal sum of Paulis	0.009	0.006	0.019	0.011	0.016	0.189
Engineered spectrum	0.009	0.008	0.015	0.012	0.079	0.076
Target: Random polynomials	Degree 1		Degree 2		Degree 3	
	Train	Test	Train	Test	Train	Test
Single-qubit Pauli	0.150	0.088	6.776	2.079	267.4	112.1
Triple-qubit Pauli	0.126	0.041	4.444	2.605	175.7	111.1
Optimal sum of Paulis	0.047	0.021	2.557	2.028	131.4	113.0
Engineered spectrum	0.037	0.028	2.225	2.033	119.8	113.0
Target: Special functions	Square-root		Exponential		Sigmoid	
	Train	Test	Train	Test	Train	Test
Single-qubit Pauli	0.637	0.187	3.032	1.069	0.084	0.030
Triple-qubit Pauli	0.325	0.104	1.654	0.733	0.044	0.018
Optimal sum of Paulis	0.139	0.073	0.715	0.436	0.017	0.008
Engineered spectrum	0.189	0.167	0.733	0.637	0.018	0.015

Table 3: Numerical results of fitting each model to three random instances of said models. For each instance, the column ‘Train’ displays the final mean squared error of the model with the target instance measured at the collocation points. The column ‘Test’ displays the final mean squared error measured at 1000 equally spaced points on the domain $[0, 2\pi]$. All values are rounded to three decimal places (so that value 0 means < 0.0005) if less than 10, or to four significant figures if at least 10.